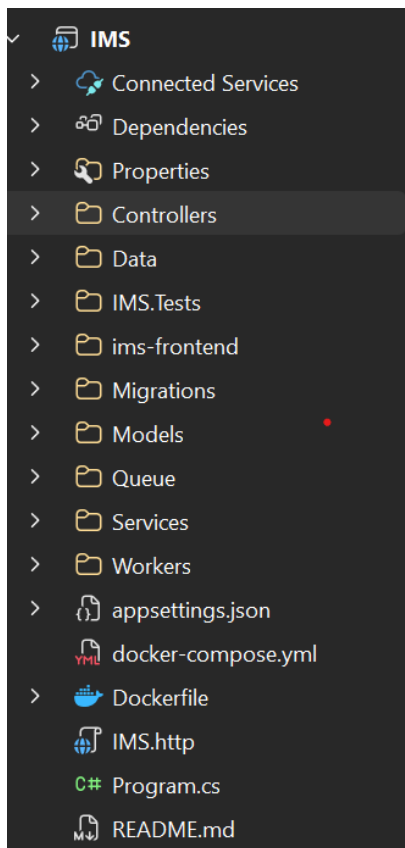


FOLDER STRUCTURE

```
IMS/
├── Controllers/
│   ├── SignalController.cs
│   ├── WorkItemController.cs
│   └── RCAController.cs
├── Models/
│   ├── Signal.cs
│   ├── WorkItem.cs
│   ├── RCA.cs
│   └── User.cs
├── Data/
│   └── AppDbContext.cs
├── Services/
│   ├── IncidentService.cs
│   ├── MongoService.cs
│   └── AuthService.cs
├── Queue/
│   └── SignalQueue.cs
├── Workers/
│   └── SignalWorker.cs
├── Program.cs
└── appsettings.json
```

```
src/
├── pages/
│   ├── Dashboard.js
│   ├── Detail.js
│   └── RCAForm.js
└── App.js
```



Incident Management System (IMS)

Overview

This project is a **Mission-Critical Incident Management System** designed to handle high-throughput system signals (10,000/sec), process them asynchronously, and manage incident lifecycle with mandatory RCA enforcement.

Architecture

Flow

```
Client → API → In-Memory Queue → Background Worker
                                   → MongoDB (Signals)
                                   → SQL Server (WorkItems)
                                   → Redis (Dashboard - optional)
```

Tech Stack

Layer	Technology
Backend	ASP.NET Core Web API
Queue	In-Memory Channel
SQL DB	SQL Server / LocalDB
NoSQL	MongoDB
Frontend	React
Container	Docker

Features

✓ High Throughput Ingestion

- Handles burst traffic using async queue (Channel)

✓ Debouncing Logic

- Prevents duplicate incidents within 10 seconds

✓ Dual Storage

- MongoDB → raw signals (audit log)
- SQL → structured incidents (source of truth)

✓ Workflow Engine

- OPEN → INVESTIGATING → RESOLVED → CLOSED

✓ Mandatory RCA

- Incident cannot be closed without RCA

✓ MTTR Calculation

- Automatically calculated from start to closure

✓ Background Processing

- Worker processes signals asynchronously

🔑 (Optional) Authentication

- JWT-based authentication
- Role-based access (Admin/User)

📄 Frontend

- Dashboard: View active incidents
- Detail Page: View incident info
- RCA Form: Submit root cause & fix

🚀 Setup Instructions

1 Clone Repo

```
git clone <your-repo>
cd IMS
```

2 Run with Docker (Recommended)

```
docker compose up --build
```

3 Run Manually

Backend

```
dotnet run
```

Frontend

```
npm start
```

4 Database Migration

```
dotnet ef migrations add Init
dotnet ef database update
```

□ Sample API

POST /api/signals

```
{
  "componentId": "RDBMS_PRIMARY",
  "payload": "Database timeout"
}
```

▣ Key Design Decisions

◆ Backpressure Handling

- Used Channel queue to prevent overload
- Worker processes signals asynchronously

◆ Resilience

- Mongo failures do not crash system
- Retry logic implemented

◆ Scalability

- Can replace Channel with Kafka
 - Redis can be added for real-time dashboard
-

Testing Strategy

- Unit tests for:
 - RCA validation
 - Debounce logic
-

Future Improvements

- Kafka integration
 - WebSocket live dashboard
 - Distributed tracing
 - Alerting integrations (Email/SMS)
-

Author

Your Name

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

```
dotnet add package Microsoft.EntityFrameworkCore.Design
```

```
dotnet add package MongoDB.Driver
```

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

```
dotnet add package Microsoft.IdentityModel.Tokens
```

MODELS

Signal.cs

```
namespace IMS.Models;

public class Signal
{
    public string ComponentId { get; set; }
    public string Payload { get; set; }
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;
}
```

WorkItem.cs

```
namespace IMS.Models;

public class WorkItem
{
    public int Id { get; set; }
    public string ComponentId { get; set; }
    public string Status { get; set; } = "OPEN";
    public string Severity { get; set; }
    public DateTime StartTime { get; set; } = DateTime.UtcNow;
    public DateTime? EndTime { get; set; }
}
```

RCA.cs

```
namespace IMS.Models;

public class RCA
{
    public int Id { get; set; }
    public int WorkItemId { get; set; }
    public string RootCause { get; set; }
    public string Fix { get; set; }
    public string Prevention { get; set; }
}
```

User.cs

```
namespace IMS.Models;

public class User
{
    public int Id { get; set; }
    public string Username { get; set; }
    public string Password { get; set; }
    public string Role { get; set; }
}
```

5. DB CONTEXT

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using IMS.Models;

namespace IMS.Data;

public class AppDbContext : DbContext
{
}
```

```

    public DbSet<WorkItem> WorkItems { get; set; }
    public DbSet<RCA> RCAs { get; set; }
    public DbSet<User> Users { get; set; }

    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }
}

```

⚡ 6. QUEUE

```

using System.Threading.Channels;
using IMS.Models;

public class SignalQueue
{
    private readonly Channel<Signal> _queue =
Channel.CreateUnbounded<Signal>();

    public async Task Enqueue(Signal signal)
    {
        await _queue.Writer.WriteAsync(signal);
    }

    public IEnumerable<Signal> ReadAll()
    {
        return _queue.Reader.ReadAllAsync();
    }
}

```

🌐 7. MONGO SERVICE (SAFE)

```

using MongoDB.Driver;
using IMS.Models;

public class MongoService
{
    private readonly IMongoCollection<Signal> _signals;

    public MongoService(IConfiguration config)
    {
        var client = new MongoClient(config["Mongo:Connection"]);
        var db = client.GetDatabase("IMS");
        _signals = db.GetCollection<Signal>("Signals");
    }

    public async Task SaveSignal(Signal signal)
    {
        try
        {
            await _signals.InsertOneAsync(signal);
        }
        catch (Exception ex)
        {
            Console.WriteLine("Mongo Error: " + ex.Message);
        }
    }
}

```

```
    }  
  }  
}
```

8. INCIDENT SERVICE

```
using IMS.Models;  
using IMS.Data;  
  
public class IncidentService  
{  
    private readonly MongoService _mongo;  
    private readonly AppDbContext _db;  
  
    private static readonly Dictionary<string, DateTime> _debounce = new();  
  
    public IncidentService(MongoService mongo, AppDbContext db)  
    {  
        _mongo = mongo;  
        _db = db;  
    }  
  
    public async Task Handle(Signal signal)  
    {  
        await _mongo.SaveSignal(signal);  
  
        var now = DateTime.UtcNow;  
  
        if (_debounce.ContainsKey(signal.ComponentId) &&  
            (now - _debounce[signal.ComponentId]).TotalSeconds < 10)  
            return;  
  
        var work = new WorkItem  
        {  
            ComponentId = signal.ComponentId,  
            Severity = signal.ComponentId.Contains("RDBMS") ? "P0" : "P2"  
        };  
  
        _db.WorkItems.Add(work);  
        await _db.SaveChangesAsync();  
  
        _debounce[signal.ComponentId] = now;  
    }  
  
    public async Task Close(int id)  
    {  
        var work = await _db.WorkItems.FindAsync(id);  
        var rca = _db.RCAs.FirstOrDefault(x => x.WorkItemId == id);  
  
        if (rca == null || string.IsNullOrEmpty(rca.RootCause))  
            throw new Exception("RCA required");  
  
        work.Status = "CLOSED";  
        work.EndTime = DateTime.UtcNow;  
  
        await _db.SaveChangesAsync();  
    }  
}
```

9. WORKER

```
public class SignalWorker : BackgroundService
{
    private readonly SignalQueue _queue;
    private readonly IServiceScopeFactory _scopeFactory;

    public SignalWorker(SignalQueue queue, IServiceScopeFactory
scopeFactory)
    {
        _queue = queue;
        _scopeFactory = scopeFactory;
    }

    protected override async Task ExecuteAsync(CancellationToken
stoppingToken)
    {
        await foreach (var signal in _queue.ReadAll())
        {
            using var scope = _scopeFactory.CreateScope();
            var service =
scope.ServiceProvider.GetRequiredService<IncidentService>();

            await service.Handle(signal);
        }
    }
}
```

10. CONTROLLERS

SignalController

```
using Microsoft.AspNetCore.Mvc;
using IMS.Models;

[ApiController]
[Route("api/signals")]
public class SignalController : ControllerBase
{
    private readonly SignalQueue _queue;

    public SignalController(SignalQueue queue)
    {
        _queue = queue;
    }

    [HttpPost]
    public async Task<IActionResult> Post(Signal signal)
    {
        await _queue.Enqueue(signal);
        return Accepted();
    }
}
```

```
    }  
}
```

WorkItemController

```
using Microsoft.AspNetCore.Mvc;  
using IMS.Data;  
  
[ApiController]  
[Route("api/workitems")]  
public class WorkItemController : ControllerBase  
{  
    private readonly AppDbContext _db;  
    private readonly IncidentService _service;  
  
    public WorkItemController(AppDbContext db, IncidentService service)  
    {  
        _db = db;  
        _service = service;  
    }  
  
    [HttpGet]  
    public IActionResult Get()  
    {  
        return Ok(_db.WorkItems.ToList());  
    }  
  
    [HttpPost("{id}/close")]  
    public async Task<IActionResult> Close(int id)  
    {  
        try  
        {  
            await _service.Close(id);  
            return Ok();  
        }  
        catch (Exception ex)  
        {  
            return BadRequest(ex.Message);  
        }  
    }  
}
```

RCAController

```
using Microsoft.AspNetCore.Mvc;  
using IMS.Data;  
using IMS.Models;  
  
[ApiController]  
[Route("api/rca")]  
public class RCAController : ControllerBase  
{  
    private readonly AppDbContext _db;  
  
    public RCAController(AppDbContext db)  
    {
```

```

        _db = db;
    }

    [HttpPost]
    public async Task<IActionResult> Add(RCA rca)
    {
        _db.RCAs.Add(rca);
        await _db.SaveChangesAsync();
        return Ok();
    }
}

```

⚙️ 11. Program.cs

```

using IMS.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("Sql")));

builder.Services.AddSingleton<SignalQueue>();
builder.Services.AddSingleton<MongoService>();
builder.Services.AddScoped<IncidentService>();

builder.Services.AddHostedService<SignalWorker>();

var app = builder.Build();

app.MapControllers();

app.Run();

```

⚙️ 12. appsettings.json (WORKING)

✓ Use LocalDB (NO ERROR)

```

{
  "ConnectionStrings": {
    "Sql":
"Server=(localdb)\\MSSQLLocalDB;Database=IMS;Trusted_Connection=True;"
  },
  "Mongo": {
    "Connection": "mongodb://localhost:27017"
  }
}

```

13. RUN DATABASE

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add Init
dotnet ef database update
```

RCA ^

GET /api/rca

POST /api/rca

Signal ^

POST /api/signals

WorkItem ^

GET /api/workitems

POST /api/workitems/{id}/close

Schemas ^

 **Swagger**
Supported by SMARTBEAR

Select a definition **IMS v1** ^

IMS 1.0 OAS 3.0
<https://localhost:44300/swagger/v1/swagger.json>

RCA ^

GET /api/rca

Parameters Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:44300/api/rca' \
  -H 'accept: */*'
```

Request URL

https://localhost:44300/api/rca

Server response

Code Details

200

Response body

```
[
  {
    "id": 1,
    "workItemId": 1,
    "rootCause": "DB overload",
    "fix": "Restarted DB",
    "prevention": "Add scaling"
  }
]
```

 Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 05 May 2026 16:46:46 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Code Details

200

Response body

```
[
  {
    "id": 1,
    "workItemId": 1,
    "rootCause": "DB overload",
    "fix": "Restarted DB",
    "prevention": "Add scaling"
  }
]
```

 Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 05 May 2026 16:46:46 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code	Description	Links
200	OK	No links

POST /api/rca



RCA

POST /api/rca

Parameters

No parameters

Request body

application/json

Edit Value Schema

```
{
  "workItemId": 1,
  "rootCause": "DB overload",
  "fix": "Restarted DB",
  "prevention": "Add scaling"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:44300/api/rca' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "workItemId": 1,
    "rootCause": "DB overload",
    "fix": "Restarted DB",
    "prevention": "Add scaling"
  }'
```

Request URL

https://localhost:44300/api/rca

Server response

Code Details

200

Response headers

Request URL

https://localhost:44300/api/rca

Server response

Code

Details

200

Response headers

date: Tue, 05 May 2026 15:13:26 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET

Responses

Code	Description	Links
200	OK	No links

Signal

^

POST

/api/signals

^

Parameters

Cancel

Reset

Signal

^

POST

/api/signals

^

Parameters

Cancel

Reset

No parameters

Request body

application/json

Edit Value

Schema

```
{  "componentId": "RDBMS_MAIN",  "payload": "DB timeout"}

```

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:44300/api/signals' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "componentId": "RDBMS_MAIN",
    "payload": "DB timeout"
  }'
```

Request URL

https://localhost:44300/api/signals

Server response

Code Details

202

Undocumented

Response headers

```
date: Tue, 05 May 2026 15:10:54 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code	Description	Links
200	OK	No links

WorkItem

GET /api/workitems

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:44300/api/workitems' \
  -H 'accept: */*' \
```

Request URL

https://localhost:44300/api/workitems

Server response

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "componentId": "RDBMS_MAIN",
    "status": "OPEN",
    "severity": "P0",
    "startTime": "2026-05-05T15:11:31.9333613",
    "endTime": null
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 05 May 2026 15:13:40 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code	Description	Links
200	OK	No links

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "componentId": "RDBMS_MAIN",
    "status": "OPEN",
    "severity": "P0",
    "startTime": "2026-05-05T15:11:31.9333613",
    "endTime": null
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 05 May 2026 15:13:40 GMT
server: Microsoft-IIS/10.0
x-powered-by: ASP.NET
```

Responses

Code	Description	Links
200	OK	No links

POST

/api/workitems/{id}/close

Parameters

Cancel

Name	Description
id * required	
integer(\$int32)	1
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:44300/api/workitems/1/close' \
  -H 'accept: */*' \
  -d ''
```

Request URL

Curl

```
curl -X 'POST' \
  'https://localhost:44300/api/workitems/1/close' \
  -H 'accept: */*' \
  -d ''
```

Request URL

```
https://localhost:44300/api/workitems/1/close
```

Server response

Code	Details
200	Response headers <pre>date: Tue, 05 May 2026 15:14:28 GMT server: Microsoft-IIS/10.0 x-powered-by: ASP.NET</pre>

Responses

Code	Description	Links
200	OK	No links